SRS Εξαμηνιαίας Εργα…
Έγγραφο

Γενική περιγραφή APIs…
PDF

Διαγράμματα SRS.vpp
Αρχείο

Εκφώνηση Εξαμηνιαίας…
PDF

swagger_apis_hypothe…
Αρχείο

bluePlug.yaml
Αρχείο

greenPlug.yaml
Αρχείο

redPlug.yaml
Αρχείο

NTUA-saasPlug-main.zip
Συμπιεσμένο αρχείο

Σκοπός μας είναι να υλοποιήσουμε την εξαμηνιαία εργασία για το μάθημα της σχολής μου "Τεχνολογίες Υπηρεσιών Λογισμικού", η οποία είναι η δημιουργία μιας εφαρμογής SaaS με microservices και dockerization. Ακολουθούν κάποιες αρχικές οδηγίες και πληροφορίες:
1. Έχω επισυνάψει όλες τις διαλέξεις του μαθήματος στα project sources. Πρέπει να τις διαβάσεις όλες από τώρα αναλυτικά, να τις αφομοιώσεις και να τις κατανοήσεις, καθώς με βάση αυτές θα υλοποιήσουμε την εργασίας. Να τις ανακαλείς πριν από κάθε απάντηση.
2. Επισυνάπτω και εδώ την εκφώνηση της εξαμηνιαίας

Εμφάνιση περισσότερων

Ενημερώθηκε μνήμη

Σκέφτηκε για 21s

1. Τι έχω καταλάβει για την εργασία

Η εργασία είναι το **saasPlug**, δηλαδή μία SaaS εφαρμογή που λειτουργεί ως ενιαίος ενδιάμεσος κόμβος για χρήστες ηλεκτρικών οχημάτων και παρόχους φορτιστών. Ο τελικός χρήστης δεν χρειάζεται να μπαίνει σε διαφορετικές εφαρμογές κάθε παρόχου· βλέπει σημεία φόρτισης από πολλούς παρόχους και μπορεί να κάνει κράτηση. Η υπηρεσία είναι δωρεάν για EV users και χρεώνεται στους παρόχους φόρτισης.

Εκφώνηση Εξαμηνιαίας Εργασίας

Η εκφώνηση ζητά υποχρεωτικά:

- **microservices architecture**
- **Docker containers**
- **GitHub repository**
- **GitHub project management**

- **UML model** με use case, activity, class, component, deployment, sequence και ER/data design
- **λειτουργική εφαρμογή**
- **τεκμηρίωση χρήσης AI εργαλείων** Εκφώνηση Εξαμηνιαίας Εργασίας

Οι βασικές λειτουργίες που πρέπει να καλύψει το σύστημα είναι:

Περιοχή	Λειτουργίες
EV user	Google login, αναζήτηση/επισκόπηση φορτιστών, κράτηση σημείου φόρτισης
Provider	εγγραφή στο saasPlug/saasCharge, δικά του analytics, προβολή τιμολογίων, πληρωμή, export logs
Operator	καθολικά analytics για όλους τους παρόχους

Η εκφώνηση σημειώνει ότι Google login, payment και export αφορούν ομάδες 5-6 ατόμων, όμως εφόσον θέλεις **Google σύνδεση**, θα την εντάξουμε στον σχεδιασμό. Για payment/export μπορούμε να τα υλοποιήσουμε σε **ελεγχόμενη απλή μορφή** - π.χ. mock payment και CSV export - ώστε να υπάρχει πλήρης κάλυψη χωρίς υπερβολική πολυπλοκότητα για ομάδα ενός ατόμου. Εκφώνηση Εξαμηνιαίας Εργασίας

2. Πώς λειτουργούν οι εξωτερικοί providers

Το saasPlug πρέπει να **καταναλώνει REST APIs** για κάθε πάροχο φόρτισης. Οι πάροχοι είναι `redPlug`, `greenPlug`, `bluePlug` και δίνονται με OpenAPI 3.x προδιαγραφές. Το γενικό έγγραφο λέει ότι τα πραγματικά ονόματα endpoints, status values και fields μπορεί να διαφέρουν ανά πάροχο, άρα χρειαζόμαστε εσωτερική κανονικοποίηση.
Γενική περιγραφή APIs παρόχων φ...

Από τα YAML αρχεία που επιθεώρησα:

Provider	List endpoint	Details/status endpoint	Reservation endpoint	Κύριο πρόβλημα
redPlug	`GET /points`	`GET /point/{pointid}`	`POST /reserve/{pointid}` ή `POST /reserve/{pointid}/{minutes}`	fields τύπου `reservatione`
greenPlug	`GET /chargingPoints`	`GET /chargingPoints/{pointid}`	`POST /chargingPoints/{pointid}/reservations`	fields τύπου `state`, `reservedUnti` request body

Provider	List endpoint	Details/status endpoint	Reservation endpoint	Κύριο πρόβλημα
bluePlug	`GET /locations`	`GET /location/{pointid}/status`	`POST /location/{pointid}/hold`	fields τύπου `chargerId`, `currentStatu` `minutes`

Άρα δεν πρέπει να γράψουμε business logic που ξέρει απευθείας τα `red/green/blue` schemas. Θέλουμε **provider adapter layer** που μετατρέπει όλα σε κοινό εσωτερικό μοντέλο, π.χ. `ChargingPointDTO`, `ChargingPointStatusDTO`, `ReservationResultDTO`.

3. Αρχιτεκτονική κατεύθυνση από τις διαλέξεις

Οι διαλέξεις δείχνουν ότι microservices σημαίνει μικρά, ανεξάρτητα, deployable services, με single responsibility, δικά τους δεδομένα και πιθανή αντιγραφή δεδομένων για να μειωθεί το coupling. 5.2 Εισαγωγή σε αρχιτεκτονικές …

Για το δικό μας σύστημα, αυτό σημαίνει:

- όχι ένα monolithic backend με όλα μέσα
- όχι shared database όπου όλα τα services κάνουν αυθαίρετα joins
- σαφή service ownership
- REST για άμεσες user-facing λειτουργίες
- RabbitMQ/events για εσωτερικά γεγονότα, analytics, billing, sync
- API Gateway για να μη φαίνονται raw service URLs/ports στο frontend

Οι σημειώσεις/διαφάνεια για την εργασία θέτουν ακριβώς τα κρίσιμα ερωτήματα: live API calls ή local DB, πώς ενημερώνονται τα σημεία, πότε καλούνται τα provider APIs, πόσα microservices, RabbitMQ και orchestration. 12.2 Συζήτηση για εργασία

Η σωστή υψηλού επιπέδου απόφαση είναι:

> **Local catalog/cache για αναζήτηση, synchronous provider calls για fresh status και reservation, events/RabbitMQ για analytics, billing και background synchronization.**

Αυτό είναι πρακτικό, εξεταστικά τεκμηριώσιμο και δεν υπερφορτώνει την εργασία.

4. Πού βρισκόμαστε τώρα

Από το repo zip:

Περιοχή	Κατάσταση
Assignment material	Υπάρχει μέσα στο repo: εκφώνηση, SRS, VPP, provider OpenAPI/YAML/Postman, διαλέξεις
Πρώτη φάση UML	Έχει γίνει ήδη σε PlantUML, κάτω από `docs/diagrams/intermediate/`

Περιοχή	Κατάσταση
UML model	Υπάρχει `saasplug_uml_model.md` με 18 PlantUML blocks
Current diagrams	Είναι χρήσιμη αρχική βάση, αλλά όχι δεσμευτική για την τελική αρχιτεκτονική
Source code	Δεν βρήκα actual implementation folders τύπου `apps/`, `services/`, `packages/`, `deploy/`
Deployment	Δεν υπάρχει ακόμα Docker Compose / Dockerfiles
AI logs	Υπάρχει πρόβλεψη φακέλου, αλλά αφού το σύστημα έχει πέσει, το αφήνουμε εκτός προς το παρόν

Άρα πρακτικά έχουμε ολοκληρωμένη **τεκμηριωτική/διαγραμματική πρώτη φάση**, αλλά δεν έχει ξεκινήσει η πραγματική υλοποίηση.

Σημαντική παρατήρηση: δεν βρήκα standalone αρχείο με όνομα τύπου **Project Notes from Lectures.docx** στο τρέχον zip ή στον `/mnt/data` φάκελο. Υπάρχουν όμως σχετικές αποφάσεις ενσωματωμένες στο `AGENTS.md` και στο `architecture_decisions.md`, καθώς και η διάλεξη `12.2 Συζήτηση για εργασία.pdf`. Αν έχεις ξεχωριστό αρχείο σημειώσεων, πρέπει να το ανεβάσεις.

5. Προτεινόμενη τελική αρχιτεκτονική υψηλού επιπέδου

Προτείνω να κινηθούμε με την εξής αρχιτεκτονική:

```
Browser / Frontend
        |
        v
API Gateway / Reverse Proxy
        |
        +--> Identity & Access Service
        +--> Provider Registry Service
        +--> Provider Sync Service
        +--> Charging Point Catalog Service
        +--> Charging Point Status Service
        +--> Reservation Service
        +--> Usage & Analytics Service
        +--> Billing & Payment Service
        |
        +--> PostgreSQL logical DBs/schemas
        +--> RabbitMQ
        |
        +--> redPlug / greenPlug / bluePlug external APIs
        +--> Google OAuth
        +--> Mock Payment Gateway
```

Γιατί αυτή η διάσπαση είναι καλή

Service	Λόγος ύπαρξης
Identity & Access	Google OAuth, users, roles
Provider Registry	εγγραφή παρόχων, API credentials, provider metadata
Provider Sync	background fetch από red/green/blue
Charging Point Catalog	normalized catalog και search
Charging Point Status	fresh status lookup και status snapshots
Reservation	reservation orchestration προς providers
Usage & Analytics	clicks, searches, reservations, exports, dashboards
Billing & Payment	invoice generation, mock payments, billing events

Δεν είναι η μικρότερη δυνατή αρχιτεκτονική, αλλά είναι **καθαρή**, αντιστοιχίζεται καλά στα ζητούμενα UML/component/sequence/deployment, και δείχνει κατανόηση microservices.

6. Τι μένει να κάνουμε

Φάση Α - Κλείδωμα προδιαγραφών

Πριν δώσουμε prompts στο Codex, πρέπει να αποφασίσουμε:

Θέμα	Απόφαση που χρειάζεται
Frontend stack	React/Vite ή Next.js
Backend stack	FastAPI ή Node/Express/Nest
DB strategy	PostgreSQL με logical DBs/schemas ανά service
Auth	Google OAuth για EV users, seeded operator, provider registration
Payment	mock/simulated payment
Map	πραγματικός χάρτης ή list/table-first UI με map placeholder
API style	REST internal APIs
AI docs	προσωρινά prompt log/manual documentation λόγω προβλήματος ailog

Η προσωπική μου σύσταση για ομάδα ενός ατόμου:

Layer	Πρόταση
Frontend	React + TypeScript + Vite
Backend	Python FastAPI microservices
DB	PostgreSQL
Messaging	RabbitMQ
Gateway	Nginx ή μικρό FastAPI/Node gateway
Docker	Docker Compose
Auth	Google OAuth
Payment	mock payment service
Docs/UML	PlantUML σε Markdown

Φάση B - Repo foundation

Θα πρέπει να δημιουργηθεί καθαρή δομή:

```
apps/
  frontend/

services/
  api-gateway/
  identity/
  provider-registry/
  provider-sync/
  charging-point-catalog/
  charging-point-status/
  reservations/
  usage-analytics/
  billing-payment/

packages/
  provider-clients/
  shared-contracts/

deploy/
  docker-compose.yml
  nginx/

docs/
  diagrams/
  assignment/
  ai/
  decisions/
```

Το υπάρχον repo έχει ήδη αρκετό documentation. Αυτό που λείπει είναι το actual product skeleton.

Φάση C - Υλοποίηση MVP

Η σειρά υλοποίησης πρέπει να είναι:

1. **Provider adapters** για red/green/blue.
2. **Normalized charging point model**.
3. **Provider registry** με API credentials.
4. **Catalog sync** και αποθήκευση local catalog.
5. **Search UI/API**.
6. **Google login**.
7. **Fresh status flow**.
8. **Reservation flow**.
9. **Usage event logging**.
10. **Analytics dashboards**.
11. **Invoices + mock payment**.
12. **CSV export logs**.
13. **Docker Compose πλήρους συστήματος**.
14. **Τελική ενοποίηση diagrams με κώδικα**.

Φάση D - GitHub Project Management

Επειδή είσαι ομάδα ενός ατόμου, δεν χρειάζεται υπερβολικό project management. Θέλουμε όμως να καλύψουμε το ζητούμενο 5%.

Πρόταση:

Sprint	Περιεχόμενο
Sprint 1	repo scaffold, Docker base, provider adapters, DB schemas
Sprint 2	catalog/search/status/reservation, Google auth
Sprint 3	analytics, billing, export, final diagrams, final QA

Στο GitHub Project θα έχουμε issues τύπου:

- `service: identity`
- `service: provider-registry`
- `service: reservations`
- `frontend`
- `docker`
- `uml`
- `docs`
- `tests`

Κάθε Codex prompt θα αντιστοιχεί σε συγκεκριμένα issues.

7. Πώς θα καλύψουμε όλα τα παραδοτέα

Παραδοτέο	Πώς θα το καλύψουμε
UML model	Θα αναθεωρήσουμε τα υπάρχοντα PlantUML ώστε να ταιριάζουν στον τελικό κώδικα
Συνέπεια μοντέλου	Θα κρατήσουμε κοινό λεξιλόγιο: ίδια DTO names, service names, endpoints παντού
GitHub source code	Καθαρό monorepo με `apps`, `services`, `packages`, `deploy`, `docs`
Deployment scripts	Dockerfiles + `docker-compose.yml` + `.env.example`
Running app	Τοπικά με ένα command μέσω Docker Compose
GitHub project management	2-3 sprints, issues, labels, milestones
AI documentation	Prompt pack, Codex outputs, decision log, limitations λόγω ailog
Provider APIs	red/green/blue adapters με normalized internal contracts
Google	OAuth login για EV users
Payment	mock payment gateway, invoice lifecycle
Export	CSV export provider usage logs

8. Κύρια αρχιτεκτονική αρχή που πρέπει να κρατήσουμε

Δεν πρέπει να κάνουμε απλώς “πολλά services” για να φαίνεται microservices. Πρέπει κάθε service να έχει σαφή ευθύνη, δικά του δεδομένα ή logical schema, σαφές API και συγκεκριμένο λόγο ύπαρξης.

Η τελική εργασία πρέπει να δείχνει:

1. **SaaS σκέψη**: πάροχοι ως πελάτες της υπηρεσίας, EV users ως δωρεάν users.
2. **Microservices σκέψη**: ανεξάρτητες ευθύνες, eventing, data ownership.
3. **API integration σκέψη**: adapters και κανονικοποίηση red/green/blue.
4. **Cloud/Docker σκέψη**: πλήρης containerized εκτέλεση.
5. **Engineering discipline**: GitHub issues, diagrams consistent with code, reproducible setup.

9. Άμεσο επόμενο βήμα

Το επόμενο βήμα δεν είναι ακόμα Codex prompt. Πρέπει πρώτα να κλειδώσουμε **τελικό MVP scope και τεχνολογικές αποφάσεις**.

Προτείνω να ξεκινήσουμε με αυτό:

> Να φτιάξουμε ένα σύντομο αλλά πλήρες **Architecture Decision Record** για το τελικό saasPlug: stack, services, data strategy, sync strategy, provider adapter strategy, auth/payment/export strategy, και τι θα θεωρείται “done” για την παράδοση.

Σκέφτηκε για 2m 12s

Συμφωνώ. Από εδώ και πέρα **δεν θεωρούμε τίποτα κλειδωμένο**. Τα υπάρχοντα UML, το `architecture_decisions.md` και οι προηγούμενες προτάσεις είναι μόνο **inputs**, όχι τελικές αποφάσεις.

Επιβεβαίωση ανάγνωσης / περιορισμός

Έχω διαθέσιμα και έχω ελέγξει:

- εκφώνηση εργασίας,
- SRS,
- γενική περιγραφή provider APIs,
- `redPlug.yaml`, `greenPlug.yaml`, `bluePlug.yaml`,
- repo zip,
- υπάρχοντα PlantUML/intermediate diagrams,
- `AGENTS.md`,
- `architecture_decisions.md`,
- διαλέξεις/project discussion materials.

Σημαντικό: **δεν εντόπισα standalone αρχείο** μέσα στο zip με όνομα `Project Notes from Lectures.docx`, παρότι το `AGENTS.md` και το `architecture_decisions.md` αναφέρουν ότι υπήρχε/χρησιμοποιήθηκε τέτοιο αρχείο. Βρήκα όμως τις σχετικές αποφάσεις που φαίνεται να προήλθαν από αυτές τις σημειώσεις. Αν υπάρχει ξεχωριστό αρχείο σημειώσεων, ανέβασέ το για να το χρησιμοποιήσουμε ως ισχυρή πηγή αλήθειας.

Σταθερά δεδομένα από την εκφώνηση

Μόνο τα παρακάτω τα θεωρώ **hard constraints** επειδή προκύπτουν άμεσα από τα αρχεία της εργασίας:

1. Το saasPlug είναι SaaS για EV users και παρόχους φόρτισης: οι χρήστες βρίσκουν/κρατούν φορτιστές από οποιονδήποτε πάροχο, ενώ οι πάροχοι δίνουν πρόσβαση στο API τους και χρεώνονται για τη χρήση της υπηρεσίας. Εκφώνηση Εξαμηνιαίας Εργασίας

2. Οι βασικές λειτουργίες περιλαμβάνουν Google login, search/overview φορτιστών, reservation, provider registration, provider analytics, invoices, payment, export logs και global analytics για operator. Εκφώνηση Εξαμηνιαίας Εργασίας
3. Η αρχιτεκτονική είναι υποχρεωτικά **microservices**, με GitHub, Docker containers, GitHub project management και τεκμηρίωση χρήσης AI. Το UML πρέπει να έχει συνέπεια ονομάτων και να περιλαμβάνει use case, activity, class, component, deployment, sequence και data design. Εκφώνηση Εξαμηνιαίας Εργασίας
4. Το λογισμικό πρέπει να καταναλώνει REST API για κάθε provider, με υποθετικούς providers `redPlug`, `greenPlug`, `bluePlug`, και οι πραγματικές προδιαγραφές βρίσκονται στα OpenAPI specs. Γενική περιγραφή APIs παρόχων φ...
5. Οι διαλέξεις/συζήτηση για την εργασία βάζουν ως ανοιχτές αρχιτεκτονικές αποφάσεις: **live API calls ή local DB**, πώς ενημερώνονται τα points, πότε καλούνται provider APIs, πόσα microservices, data sync, RabbitMQ και orchestration. 12.2 Συζήτηση για εργασία

Διαδικασία κλειδώματος προδιαγραφών

Θα πάμε με σειρά και σε κάθε βήμα θα κρατάμε:

Πεδίο	Περιεχόμενο
Απόφαση	Τι κλειδώνουμε
Εναλλακτικές	Τι απορρίψαμε
Λόγος	Γιατί είναι σωστό για την εργασία
Επιπτώσεις	Πώς επηρεάζει code, UML, Docker, GitHub
Status	`proposed`, `accepted`, `rejected`, `deferred`

Δεν θα περάσουμε σε Codex prompts μέχρι να έχουμε κλειδώσει τις βασικές αποφάσεις.

Προτεινόμενη σειρά αποφάσεων

1. **Scope / MVP ανά actor**
   - τι ακριβώς κάνει EV user,
   - τι κάνει provider admin,
   - τι κάνει saasPlug operator,
   - τι υλοποιούμε πλήρως και τι mockάρουμε.
2. **Provider API strategy**
   - απευθείας live calls,
   - local catalog/cache,
   - hybrid,
   - provider adapters,
   - normalized internal DTOs.
3. **Data management strategy**
   - database-per-service,

- shared PostgreSQL container με logical DBs/schemas,
- event-synced read models,
- τι αποθηκεύεται και τι όχι.

4. **Microservices boundaries**
   - πόσα services,
   - ποια είναι πραγματικά απαραίτητα,
   - τι αποφεύγουμε ως overengineering.
5. **Synchronous vs asynchronous flows**
   - reservation,
   - status refresh,
   - provider catalog sync,
   - analytics,
   - billing,
   - RabbitMQ usage.
6. **Authentication / authorization**
   - Google login,
   - provider admin login,
   - operator access,
   - roles/JWT/session model.
7. **Billing, payment, export**
   - τι σημαίνει invoice,
   - πότε χρεώνεται provider,
   - mock payment ή πραγματική ενσωμάτωση,
   - CSV export/logs.
8. **Frontend scope**
   - pages,
   - dashboards,
   - map vs table/list,
   - login flows,
   - minimum usable UX.
9. **Internal REST API contracts**
   - public routes,
   - service APIs,
   - DTOs,
   - error handling.
10. **Persistence model**
    - entities ανά service,
    - ownership,
    - duplicates/read models,
    - IDs and references.
11. **Docker/deployment**
    - Docker Compose,
    - gateway,
    - ports,

- networks,
- volumes,
- env/secrets.

12. **GitHub/project management**
    - repo structure,
    - issues,
    - milestones/sprints,
    - branches/commits.
13. **UML regeneration strategy**
    - υπάρχοντα diagrams ως draft,
    - τελικό UML μετά ή παράλληλα με τον κώδικα,
    - consistency checks.
14. **AI documentation strategy**
    - αφού το `ailog` έχει πέσει,
    - πώς θα τεκμηριώνουμε prompts, outputs, decisions, limitations.

Spec 0 - Διαδικαστικός κανόνας

Προτείνω να κλειδώσουμε πρώτα αυτό:

> **Spec 0:** Τα υπάρχοντα diagrams και ADRs δεν δεσμεύουν την τελική αρχιτεκτονική. Θα χρησιμοποιηθούν μόνο ως ιστορικό και ως checklist. Οι τελικές προδιαγραφές θα κλειδώνονται ξανά μία προς μία σε αυτή τη συνομιλία. Το repo και τα Codex prompts θα ακολουθούν αποκλειστικά τις αποφάσεις που θα σημειώνουμε ως `accepted`.

Status: proposed

Αν συμφωνείς, προχωράμε στο **Spec 1 - Scope / MVP ανά actor**.

Project Notes from Lect…
Έγγραφο

Να διαβάσεις αυτό.
Έπειτα, πάμε να τα αποφασίσουμε όλα από την αρχή.

Σκέφτηκε για 46s ›

SPEC-2 - Provider API & Data Acquisition Strategy

Σκοπός του SPEC-2 είναι να κλειδώσουμε **πώς θα παίρνουμε, αποθηκεύουμε, συγχρονίζουμε και χρησιμοποιούμε τα δεδομένα των providers**.

Η διάλεξη για την εργασία θέτει ρητά τα ανοιχτά αρχιτεκτονικά θέματα: **live API calls ή local DB**, αν υπάρχει local DB πώς ενημερώνονται τα points, πότε γίνονται API calls, πόσα microservices, data sync strategy, RabbitMQ και orchestration.

12.2 Συζήτηση για εργασία Οι σημειώσεις σου δίνουν ισχυρή κατεύθυνση: δεν κάνουμε συνεχώς calls στους providers για όλα· κρατάμε αποθήκη δεδομένων σημείων, περιοδικά συγχρονισμένη, και κάνουμε live calls κυρίως για **status** και **reservation**.
Project Notes from Lectures

SPEC-2A - High-level strategy

Proposed decision

Χρησιμοποιούμε **hybrid provider strategy**:

Λειτουργία	Στρατηγική
Charging point catalog/list	Local DB, ενημερώνεται περιοδικά από provider APIs
Search/filter	Από local aggregated catalog
Point details	Από local catalog + optional fresh status
Live status	Real-time provider API call μέσω backend, με short cache
Reservation	Real-time provider API call μέσω backend, χωρίς queue μπροστά
Analytics/billing	Από δικά μας stored events/records, όχι live από providers

Rationale

Αυτό υλοποιεί ακριβώς τη λογική των σημειώσεων:

- τα list/catalog calls είναι τα πιο “βαριά”, άρα δεν τα κάνουμε για κάθε χρήστη,
- τα status και reservation είναι τα δύο απαραίτητα real-time calls,
- το frontend δεν καλεί απευθείας provider APIs,
- το σύστημα αποκτά δικά του δεδομένα για search, analytics και billing.

Status

`proposed`

SPEC-2B - Provider-specific microservices

Proposed decision

Θα έχουμε **ένα provider integration microservice ανά provider**:

Service	Ευθύνη
`red-provider-service`	integration με redPlug API
`green-provider-service`	integration με greenPlug API
`blue-provider-service`	integration με bluePlug API

Κάθε provider service θα ξέρει μόνο:

- το external API schema του provider,
- τα external endpoints,
- πώς μετατρέπεται η απάντηση σε normalized internal model,
- πώς γίνεται status lookup,
- πώς γίνεται reservation.

Τα υπόλοιπα services δεν πρέπει να ξέρουν αν το πεδίο λέγεται `pointid`, `id`, `chargerId`, `state`, `currentStatus`, κ.λπ.

Για νέο provider

Αν προστεθεί νέος provider, προστίθεται νέο provider service:

```
yellow-provider-service
```

χωρίς να χρειαστεί να αλλάξει όλο το σύστημα.

Status

`proposed`

SPEC-2C - Aggregated catalog service

Proposed decision

Θα υπάρχει ξεχωριστό service:

```
charging-catalog-service
```

ή:

```
catalog-aggregator-service
```

Το service αυτό κρατά το **ενιαίο normalized catalog** όλων των charging points από red/green/blue.

Ευθύνες

Ευθύνη	Περιγραφή
Store normalized points	αποθηκεύει όλα τα σημεία σε κοινό schema
Search/filter API	εξυπηρετεί το frontend για αναζήτηση
Provider filtering	φιλτράρισμα ανά provider
Staleness tracking	κρατά `last_catalog_sync_at`, `last_status_checked_at`
Missing data handling	εμφανίζει σωστά missing price/status
Manual sync visibility	δείχνει πότε έγινε τελευταίος συγχρονισμός

Internal ID strategy

Κλειδώνουμε ότι το εσωτερικό ID ενός φορτιστή δεν είναι απλώς το external provider ID.

Χρησιμοποιούμε:

```
provider_code + external_point_id
```

Παράδειγμα:

```
redPlug:12345
greenPlug:12345
bluePlug:12345
```

ώστε να μην έχουμε collisions.

Status

`proposed`

SPEC-2D - Catalog synchronization policy

Proposed decision

Ο συγχρονισμός του catalog γίνεται **περιοδικά** και **χειροκίνητα για demo/debug**.

Trigger	Απόφαση
Automatic sync	Ναι
Default frequency	1 φορά την ημέρα
Manual sync from operator dashboard	Ναι
Sync on first system startup	Ναι, αν το catalog είναι άδειο

Trigger	Απόφαση
Retry policy	Ναι, λίγες επαναλήψεις με backoff
Live sync per user search	Όχι

Upsert strategy

Επιλέγουμε την απλή και robust πολιτική:

> Κάθε provider sync φέρνει όλα τα σημεία του provider και το catalog service κάνει **full upsert with change detection**.

Δηλαδή:

- αν το point υπάρχει, ενημερώνεται,
- αν είναι νέο, εισάγεται,
- αν λείπει από νέο sync, δεν διαγράφεται άμεσα· μαρκάρεται ως `stale` ή `inactive`,
- κρατάμε timestamp τελευταίας εμφάνισης.

Αποφεύγουμε να υπολογίζει κάθε provider service πολύπλοκα diffs. Το diff/change detection γίνεται κεντρικά στο catalog service.

Status

`proposed`

SPEC-2E - RabbitMQ usage για provider sync

Proposed decision

Χρησιμοποιούμε RabbitMQ **μόνο για ασύγχρονα sync/background flows**, όχι για live frontend actions.

Για catalog sync:

```
scheduler/operator
        |
        v
RabbitMQ: provider.sync.requested
        |
        +--> red-provider-service
        +--> green-provider-service
        +--> blue-provider-service
                |
                v
RabbitMQ: provider.points.fetched
                |
                v
catalog-aggregator-service
                |
```

```
                v
catalog database
```

Δεν χρησιμοποιούμε RabbitMQ για:

Flow	Απόφαση
Search request	Όχι
View details request	Όχι
Live status request	Όχι
Reservation request	Όχι
Login/auth	Όχι

Rationale

Οι σημειώσεις λένε ότι το messaging δεν πρέπει να χρησιμοποιηθεί για αιτήσεις του frontend και ότι ο κλασικός λόγος χρήσης του είναι συγχρονισμός βάσεων/δεδομένων.

Project Notes from Lectures

Status

`proposed`

SPEC-2F - Live status strategy

Proposed decision

Το live status γίνεται μέσω:

```
frontend
  -> api-gateway
  -> charging-status-service
  -> provider-specific service
  -> external provider API
```

Το `charging-status-service`:

- δέχεται `internal_point_id`,
- βρίσκει provider και external ID,
- ελέγχει short cache,
- αν το cache είναι fresh, επιστρέφει cached status,
- αλλιώς καλεί το αντίστοιχο provider service,
- αποθηκεύει `last_known_status` και `last_status_checked_at`.

Cache TTL

Προτεινόμενη απλή τιμή:

```
60 seconds
```

Αυτό είναι αρκετά μικρό για να φαίνεται “live” στο demo, αλλά αποφεύγει συνεχείς επαναλαμβανόμενες κλήσεις.

Status values

Κλειδώνουμε εσωτερικό normalized enum:

```
available
charging
reserved
malfunction
offline
unknown
```

Αν provider επιστρέψει άγνωστο ή missing status, κάνουμε map σε:

```
unknown
```

Status

`proposed`

SPEC-2G - Reservation strategy

Proposed decision

Η κράτηση γίνεται **real-time** και **synchronously**.

Flow:

```
frontend
  -> api-gateway
  -> reservation-service
  -> provider-specific service
  -> external provider reservation endpoint
```

Κανόνας

Δεν βάζουμε queue πριν από το reservation call.

Ο λόγος είναι ότι ο χρήστης χρειάζεται άμεση απάντηση:

- κράτηση επιτυχής,
- κράτηση αποτυχημένη,
- provider unavailable,
- invalid point/status,

- timeout.

Τι αποθηκεύουμε

Το `reservation-service` αποθηκεύει:

Field	Περιγραφή
reservation id	εσωτερικό ID
user id	EV user
provider code	red/green/blue
external point id	provider point
internal point id	normalized ID
requested duration	αν υπάρχει
provider response	normalized result
status	`successful`, `failed`, `pending_external_unknown`
reservation end time	αν δοθεί
error reason	αν αποτύχει
created at	timestamp

Μετά την καταγραφή, τα δεδομένα μπορούν να πάνε σε analytics/billing με event ή περιοδικό sync. Δεν επηρεάζεται η άμεση απάντηση στον χρήστη.

Status

`proposed`

SPEC-2H - Missing values, timeouts, fallback

Proposed decision

Κλειδώνουμε απλή αλλά συστηματική πολιτική.

Περίπτωση	Συμπεριφορά
Missing price	Εμφάνιση `Price unavailable`
Previously known price exists	Μπορούμε να δείξουμε `Last known price` με timestamp
Missing status	`unknown`

Περίπτωση	Συμπεριφορά
Provider timeout στο status	επιστροφή last known status, αν υπάρχει, με warning
Provider timeout στο reservation	αποτυχημένη προσπάθεια, καταγραφή error
Provider API 400/404/500	normalized error response
Sync failure	καταγραφή failed sync run, retry, διατήρηση παλιού catalog

Frontend messages

Δεν εμφανίζουμε raw provider errors στον χρήστη. Εμφανίζουμε καθαρά μηνύματα:

```
Status temporarily unavailable. Showing last known information.
```

```
Reservation could not be completed because the provider did not respond.
```

Status

`proposed`

SPEC-2I - External API visibility / gateway rule

Proposed decision

Κανένα browser request δεν πηγαίνει απευθείας:

- σε provider API,
- σε internal microservice port,
- σε raw Docker service URL.

Όλα περνούν από:

```
frontend -> gateway/reverse proxy -> backend services
```

Αυτό ταιριάζει με τις σημειώσεις για το πρόβλημα του browser inspect και την ανάγκη να μη φαίνονται απευθείας URLs/ports. Project Notes from Lectures

Status

`proposed`

SPEC-2 - Συνολική πρόταση

Proposed accepted decision

Πεδίο	Απόφαση
Overall strategy	Hybrid
Catalog/list points	Local aggregated catalog
Catalog update	Periodic full upsert with change detection
Default sync frequency	Daily
Manual sync	Ναι, από operator
Provider services	Ένα ανά provider
Aggregator/catalog service	Ναι
Status	Real-time μέσω backend + 60s cache
Reservation	Real-time synchronous, no queue
RabbitMQ	Μόνο για background sync/background propagation
Missing values	explicit fallback policy
Frontend direct provider calls	Απαγορεύονται
API gateway/reverse proxy	Υποχρεωτικό στο runtime design

Status: `proposed`

Απόφαση που χρειάζεται τώρα

Να κλειδώσουμε το **SPEC-2** ως έχει ή να αλλάξουμε κάποιο από τα παρακάτω;

Θέμα	Τρέχουσα πρόταση
Provider strategy	Hybrid
Provider services	3 services, ένα ανά provider
Catalog storage	Local aggregated catalog
Catalog sync	Daily + manual + first-start
Sync policy	Full upsert with change detection
RabbitMQ	Μόνο για background sync
Status cache	60 seconds

Θέμα	Τρέχουσα πρόταση
Reservation	Real-time, no queue
Frontend provider calls	Όχι, όλα μέσω gateway/backend

Σκέφτηκε για 55s

SPEC-3 - Microservices Boundaries

Σκοπός του **SPEC-3** είναι να κλειδώσουμε **ποια microservices θα υπάρχουν**, ποια ευθύνη έχει το καθένα, τι δεν θα γίνει ξεχωριστό service, και πώς αποφεύγουμε overengineering.

Η εκφώνηση κάνει υποχρεωτική τη microservices αρχιτεκτονική και ζητά Docker deployment, GitHub source/deployment scripts και UML consistency ανά component/API/data model. Εκφώνηση Εξαμηνιαίας Εργασίας Οι σημειώσεις σου επίσης δείχνουν σαφή κατεύθυνση για provider-specific services, aggregator, ξεχωριστή λογική status/reservation/statistics και χρήση messaging κυρίως για συγχρονισμό/aggregation, όχι για live frontend αιτήσεις. Project Notes from Lectures

---

SPEC-3A - Αρχή διάσπασης

Proposed decision

Η διάσπαση σε microservices θα γίνει με βάση **business responsibility**, όχι με βάση τεχνικά layers.

Δηλαδή δεν θα έχουμε services τύπου:

```
controller-service
repository-service
database-service
```

Θα έχουμε services τύπου:

```
auth-service
catalog-service
reservation-service
analytics-service
billing-service
```

Rationale

Αυτό ταιριάζει καλύτερα με την έννοια microservices: κάθε service έχει μία σαφή επιχειρησιακή ευθύνη, δικό του API, και κατά προτίμηση δικά του δεδομένα/read model.

Status

`proposed`

SPEC-3B - Τελικό προτεινόμενο service set

Proposed runtime components

Component	Τύπος	Business microservice;	Περιγραφή
`frontend`	Web app	Όχι	UI για EV users, provider admins, operator
`api-gateway`	Gateway/reverse proxy	Όχι	Ενιαίο entrypoint, routing, hiding internal ports
`auth-service`	Backend service	Ναι	Google login, email/password login, JWT, roles
`provider-registry-service`	Backend service	Ναι	Provider registration, API config, provider metadata
`red-provider-service`	Backend service	Ναι	Adapter/integration για redPlug
`green-provider-service`	Backend service	Ναι	Adapter/integration για greenPlug
`blue-provider-service`	Backend service	Ναι	Adapter/integration για bluePlug
`catalog-service`	Backend service	Ναι	Aggregated normalized charging-point catalog, search/filter
`status-service`	Backend service	Ναι	Live/cached charger status lookup

Component	Τύπος	Business microservice;	Περιγραφή
`reservation-service`	Backend service	Ναι	Reservation flow, reservation records, reservation events
`analytics-service`	Backend service	Ναι	Provider/operator analytics, usage logs, CSV export
`billing-service`	Backend service	Ναι	Pricing rules, billing records, invoices, mock payment
`postgres`	Infrastructure	Όχι	DBMS, με logical DBs/schemas ανά service
`rabbitmq`	Infrastructure	Όχι	Async sync/events για catalog/statistics/billing
optional `scheduler-worker`	Worker	Όχι/part of catalog	Periodic sync trigger, μπορεί να ανήκει στο `catalog-service`

Προτεινόμενος αριθμός backend business services: 9.

Αυτό είναι αρκετά πλούσιο για να φαίνεται σωστά η microservices αρχιτεκτονική, αλλά όχι υπερβολικό τύπου 15-20 services.

Status

`proposed`

SPEC-3C - API Gateway

Proposed decision

Το `api-gateway` είναι υποχρεωτικό runtime component, αλλά όχι business microservice.

Ευθύνες

Responsibility	Απόφαση
Single public backend entrypoint	Ναι
Hide internal service ports	Ναι
Route `/api/auth/*` to `auth-service`	Ναι
Route `/api/catalog/*` to `catalog-service`	Ναι
Route `/api/reservations/*` to `reservation-service`	Ναι

Responsibility	Απόφαση
Route `/api/analytics/*` to `analytics-service`	Ναι
Route `/api/billing/*` to `billing-service`	Ναι
Implement business logic	Όχι
Direct provider API calls	Όχι

Rationale

Οι σημειώσεις επισημαίνουν ότι δεν πρέπει στο browser inspect να φαίνονται raw URLs/ports και ότι είναι καλή ιδέα να περνούν όλα από gateway/reverse proxy.

Project Notes from Lectures

Status

`proposed`

SPEC-3D - Auth Service

Proposed decision

Θα υπάρχει ξεχωριστό:

```
auth-service
```

Ευθύνες

Responsibility	Περιγραφή
Google login	EV user OAuth login
Email/password login	απλό credential login
JWT issuing	access token με user id/role
Role management	`EV_USER`, `PROVIDER_ADMIN`, `OPERATOR`
Password hashing	για email/password users
Seed operator account	αρχικός operator
Token validation endpoint	για gateway/services αν χρειαστεί

Δεν κάνει

Δεν κάνει	Ανήκει σε
Provider business registration	`provider-registry-service`
Provider analytics authorization logic	`analytics-service`, με role/provider checks
Billing authorization logic	`billing-service`, με role/provider checks

Rationale

Το auth αξίζει ξεχωριστό service επειδή έχουμε τρεις actor categories και δύο login methods: Google και email/password/JWT.

Status

`proposed`

SPEC-3E - Provider Registry Service

Proposed decision

Θα υπάρχει ξεχωριστό:

```
provider-registry-service
```

Ευθύνες

Responsibility	Περιγραφή
Register provider	επωνυμία, provider code/type, contact info
Store provider API config	base URL, API token/team key όπου χρειάζεται
Provider activation status	active/inactive
Provider metadata	display name, color/code, created date
Provider lookup API	δίνει config σε provider services/internal services
Basic config update	provider/operator controlled update

Δεν κάνει

Δεν κάνει	Ανήκει σε
Fetch charging points	provider-specific services
Search points	`catalog-service`

Δεν κάνει	Ανήκει σε
Billing	`billing-service`
Analytics	`analytics-service`

Rationale

Το SRS περιγράφει provider registration όπου ο πάροχος καταχωρεί τα απαιτούμενα στοιχεία ώστε το saasCharge/saasPlug να επικοινωνεί με το API του. SRS Εξαμηνιαίας Εργασίας Άρα έχει νόημα να υπάρχει καθαρό service που κατέχει provider metadata και API configuration.

Status

`proposed`

SPEC-3F - Provider Integration Services

Proposed decision

Θα υπάρχουν τρία provider-specific services:

```
red-provider-service
green-provider-service
blue-provider-service
```

Κοινή ευθύνη κάθε provider service

Responsibility	Περιγραφή
Call provider list endpoint	φέρνει σημεία provider
Normalize provider point	μετατρέπει σε κοινό internal DTO
Call provider status endpoint	φέρνει status για συγκεκριμένο point
Normalize status	map σε internal enum
Call reservation endpoint	κάνει πραγματική κράτηση
Normalize reservation response	επιστρέφει κοινό reservation result
Provider-specific errors	μετατροπή σε normalized errors

Δεν κάνει

Δεν κάνει	Ανήκει σε
Store global catalog	`catalog-service`
Decide billing	`billing-service`
Own user reservations	`reservation-service`
Expose public browser API	gateway/backend only

Persistent DB;

Προτείνω:

> Τα provider services να είναι κυρίως **stateless adapters**. Δεν χρειάζονται δική τους persistent DB στο MVP.

Μπορούν να κρατούν μόνο logs/temporary diagnostics αν χρειαστεί. Το normalized catalog ανήκει στο `catalog-service`.

Rationale

Οι σημειώσεις λένε ότι κάθε provider θα έχει δικό του microservice που διαβάζει και ότι για νέο provider προστίθεται νέο microservice. Επίσης προβλέπουν aggregated microservice πάνω από τα provider-specific services. Project Notes from Lectures

Status

`proposed`

SPEC-3G - Catalog Service

Proposed decision

Θα υπάρχει:

```
catalog-service
```

Το `catalog-service` είναι το **aggregator** των provider services.

Ευθύνες

Responsibility	Περιγραφή
Store normalized charging points	ενιαίος κατάλογος red/green/blue
Search/filter API	εξυπηρετεί EV user search
Provider-specific filtering	provider admin/operator filters

Responsibility	Περιγραφή
Point details	static/catalog details
Staleness tracking	`last_catalog_sync_at`, `last_seen_at`
First-start sync trigger	αν ο κατάλογος είναι άδειος
Manual sync trigger	από operator
Consume provider sync results	μέσω RabbitMQ ή internal calls
Upsert with change detection	full provider list → insert/update/stale

Δεν κάνει

Δεν κάνει	Ανήκει σε
Live status truth	`status-service`
Reservation	`reservation-service`
Provider API schema handling	provider-specific services
Analytics aggregation	`analytics-service`

Rationale

Το SRS απαιτεί search/overview με φίλτρα και εμφάνιση διαθέσιμων πληροφοριών/τελευταίας ενημέρωσης status. SRS Εξαμηνιαίας Εργασίας Οι σημειώσεις σου επίσης λένε ότι πρέπει να έχουμε αποθήκη σημείων, ώστε να μη γίνονται μαζικές κλήσεις στους providers για κάθε πελάτη. Project Notes from Lectures

Status

`proposed`

SPEC-3H - Status Service

Proposed decision

Θα υπάρχει ξεχωριστό:

```
status-service
```

Ευθύνες

Responsibility	Περιγραφή
Resolve internal point id	βρίσκει provider/external id μέσω catalog
Live status lookup	καλεί provider-specific service
Status cache	TTL 5 λεπτά
Last known status	αποθήκευση τελευταίου γνωστού status
Timeout fallback	επιστρέφει last known status όπου υπάρχει
Publish status events	optional για analytics

Δεν κάνει

Δεν κάνει	Ανήκει σε
Search/list points	`catalog-service`
Reservation	`reservation-service`
Provider-specific HTTP details	provider-specific services

Rationale

Οι σημειώσεις αναφέρουν ξεχωριστά την κατάσταση σημείων ως microservice με κλήσεις προς provider APIs, ενώ έχουμε ήδη κλειδώσει real-time backend-mediated status με 5λεπτο cache. Project Notes from Lectures

Status

`proposed`

SPEC-3I - Reservation Service

Proposed decision

Θα υπάρχει ξεχωριστό:

```
reservation-service
```

Ευθύνες

Responsibility	Περιγραφή
Accept reservation request	από EV user μέσω gateway
Validate user/point	user, point, provider, duration

Responsibility	Περιγραφή
Call provider-specific reservation	synchronous real-time call
Store reservation record	success/failure
Store provider response	normalized
Publish reservation events	προς analytics/billing
Return result to user	immediate response

Δεν κάνει

Δεν κάνει	Ανήκει σε
Queue before provider reservation	δεν γίνεται
Compute analytics dashboards	`analytics-service`
Generate invoices	`billing-service`
Provider-specific endpoint handling	provider-specific services

Rationale

Το SRS λέει ότι ο χρήστης αιτείται κράτηση, λαμβάνει χρόνο λήξης αν ικανοποιηθεί, και το σύστημα καταγράφει τη χρέωση που αναλογεί στον provider. SRS Εξαμηνιαίας Εργασίας Οι σημειώσεις σου επίσης ξεκαθαρίζουν ότι messaging για κράτηση δεν παίζει και ότι οι αιτήσεις κατάστασης/κράτησης είναι real-time. Project Notes from Lectures

Status

`proposed`

SPEC-3J - Analytics Service

Proposed decision

Θα υπάρχει ξεχωριστό:

```
analytics-service
```

Ευθύνες

Responsibility	Περιγραφή
Consume usage events	searches, views/clicks, status checks, reservations
Maintain analytics read model	ανεξάρτητη βάση/read tables
Provider analytics API	own analytics για provider admin
Global analytics API	operator analytics
CSV export logs	export own provider usage logs
Filter by provider/date	βασικά φίλτρα
Continue working if reservation service is down	διαβάζει από δική του βάση

Δεν κάνει

Δεν κάνει	Ανήκει σε
Own reservation truth	`reservation-service`
Own billing truth	`billing-service`
Call provider APIs	provider/status/catalog services
Real-time blocking flows	όχι

Rationale

Το SRS ζητά provider analytics, global analytics και export logs. SRS Εξαμηνιαίας Εργασίας Οι σημειώσεις λένε ότι τα statistics πρέπει να είναι άλλο microservice, να παίρνουν στοιχεία κρατήσεων και κλικ, και να μπορούν να θεαθούν ακόμη και αν πέσει το reservation service. Project Notes from Lectures

Status

`proposed`

SPEC-3K - Billing Service

Proposed decision

Θα υπάρχει ξεχωριστό:

```
billing-service
```

Ευθύνες

Responsibility	Περιγραφή
Pricing rules	editable constants/rules ανά provider
Billing records	charges από reservations/clicks αν αποφασιστεί
Invoice generation	invoice ανά provider/period
Invoice view API	provider admin/operator
Mock payment	pay whole invoice
Payment status	paid/unpaid
Billing event consumption	reservation/usage events
Reconciliation-friendly design	eventual consistency αποδεκτή

Δεν κάνει

Δεν κάνει	Ανήκει σε
Real payment gateway integration	out of scope
Partial invoice payment	out of scope
Analytics dashboards	`analytics-service`
Reservation execution	`reservation-service`

Rationale

Το SRS προβλέπει view invoice και pay invoice, ενώ η κράτηση παράγει χρέωση προς τον provider. SRS Εξαμηνιαίας Εργασίας Οι σημειώσεις σου λένε ότι το billing μπορεί να είναι ασύγχρονο/B2B, να μη χρειάζεται όλα τα δεδομένα αμέσως, και ότι πληρώνεται όλο το ποσό, όχι κομμάτια του. Project Notes from Lectures

Status

`proposed`

SPEC-3L - Τι δεν γίνεται ξεχωριστό service

Για να αποφύγουμε overengineering:

Δεν γίνεται ξεχωριστό service	Πού ανήκει
`pricing-service`	μέσα στο `billing-service`

Δεν γίνεται ξεχωριστό service	Πού ανήκει
`payment-service`	mock payment μέσα στο `billing-service`
`export-service`	CSV export μέσα στο `analytics-service`
`search-service`	search μέσα στο `catalog-service`
`map-service`	frontend + catalog API
`notification-service`	out of scope
`load-balancer-service`	out of scope
`ai-log-service`	out of scope
`admin-service`	operator features μοιράζονται στα relevant services
`provider-dashboard-service`	frontend role views πάνω στα existing APIs

Status

`proposed`

SPEC-3M - Communication matrix

Synchronous REST/internal HTTP

From	To	Γιατί
frontend	api-gateway	public app access
api-gateway	auth-service	login/token flows
api-gateway	catalog-service	search/details
api-gateway	status-service	status lookup
api-gateway	reservation-service	reservation request
api-gateway	analytics-service	dashboards/export
api-gateway	billing-service	invoices/payment
status-service	catalog-service	resolve point/provider
status-service	provider services	live status
reservation-service	catalog-service	resolve point/provider

From	To	Γιατί
reservation-service	provider services	reservation call
provider services	provider-registry-service	load provider config

Asynchronous RabbitMQ/events

Producer	Event	Consumer
catalog-service/scheduler	`provider.sync.requested`	provider services
provider services	`provider.points.fetched`	catalog-service
catalog-service	`point.viewed` / `search.performed`	analytics-service
status-service	`point.status.checked`	analytics-service
reservation-service	`reservation.succeeded` / `reservation.failed`	analytics-service, billing-service
billing-service	`invoice.paid`	analytics-service optional

Status

`proposed`

SPEC-3N - Προτεινόμενη δομή repo

Αυτό δεν είναι ακόμα implementation prompt, αλλά για consistency:

```
apps/
  frontend/

services/
  api-gateway/
  auth-service/
  provider-registry-service/
  red-provider-service/
  green-provider-service/
  blue-provider-service/
  catalog-service/
  status-service/
  reservation-service/
  analytics-service/
  billing-service/

packages/
  shared-contracts/
  provider-normalization/
```

```
deploy/
  docker-compose.yml
  nginx/

docs/
  specs/
  decisions/
  diagrams/
  ai/
```

Status

`proposed`

SPEC-3 - Συνολική πρόταση

Proposed accepted decision

Πεδίο	Απόφαση
Business microservices	`auth`, `provider-registry`, 3 provider services, `catalog`, `status`, `reservation`, `analytics`, `billing`
Gateway	Ναι, αλλά όχι business microservice
Frontend	ξεχωριστό app/container
Provider services	stateless adapters, ένα ανά provider
Aggregator	`catalog-service`
Status	ξεχωριστό service με 5λεπτο cache
Reservation	ξεχωριστό service, sync provider call, no queue before reservation
Analytics	ξεχωριστό service με δικό του read model
Billing	ξεχωριστό service με pricing/invoices/mock payment
Pricing	μέσα στο billing, όχι ξεχωριστό service
Export logs	μέσα στο analytics
RabbitMQ	background sync/events μόνο
DB ownership	θα κλειδωθεί στο SPEC-4

Πεδίο	Απόφαση
Event contracts	θα κλειδωθούν στο SPEC-5
Analytics metrics	θα κλειδωθούν στο SPEC-6
Billing model	θα κλειδωθεί στο SPEC-7

Status: `proposed`

Απόφαση που χρειάζεται τώρα

Να κλειδώσουμε το **SPEC-3** με αυτό το service split;

Τα βασικά σημεία προς επιβεβαίωση είναι:

Θέμα	Τρέχουσα πρόταση
Provider services	3 ξεχωριστά services
Catalog aggregator	ξεχωριστό `catalog-service`
Status	ξεχωριστό `status-service`
Reservation	ξεχωριστό `reservation-service`
Analytics	ξεχωριστό `analytics-service`
Billing	ξεχωριστό `billing-service`
Provider registry	ξεχωριστό `provider-registry-service`
Pricing service	όχι, μέσα στο billing
Export service	όχι, μέσα στο analytics
Payment service	όχι, mock μέσα στο billing

Ενημερώθηκε μνήμη

Thought for 32s

SPEC-7 - Accepted με τροποποίηση στο 7F

Συμφωνούμε και το κλειδώνουμε:

> **Μόνο ο operator/admin μπορεί να εκδώσει επίσημο invoice. Οι providers μπορούν να βλέπουν και να πληρώνουν issued invoices. Προαιρετικά μπορούν να αιτούνται έκδοση invoice, αλλά όχι να το εκδίδουν οι ίδιοι.**

Αυτό είναι συμβατό με την εκφώνηση/SRS αν το χειριστούμε προσεκτικά. Η εκφώνηση ζητά “θέαση τιμολογίου χρήσης προς παρόχους” και “πληρωμή τιμολογίου από παρόχους”, όχι απαραίτητα provider-side invoice issuance. Εκφώνηση Εξαμηνιαίας Εργασίας Το SRS λέει ότι εκδίδεται τιμολόγιο προς τον πάροχο και εμφανίζεται στην οθόνη, ενώ ο

χρήστης μπορεί να αιτηθεί πληρωμή· επίσης αναφέρει ότι ο provider μπορεί να αιτηθεί έκδοση τιμολογίου. SRS Εξαμηνιαίας Εργασίας

Η πιο ασφαλής ερμηνεία για να καλύψουμε και τα δύο είναι:

Action	Actor
Generate draft invoice	`billing-service` / operator
Issue official invoice	`OPERATOR`
Request invoice issuance	`PROVIDER_ADMIN`, optional
View issued invoice	`PROVIDER_ADMIN`, `OPERATOR`
Pay issued invoice	`PROVIDER_ADMIN`
Cancel/void invoice	`OPERATOR` only

Άρα το **SPEC-7 γίνεται accepted**.

SPEC-8 - Authentication & Authorization

Σκοπός του SPEC-8 είναι να κλειδώσουμε login methods, roles, access control, token strategy και seeded users.

SPEC-8A - Auth service ownership

Proposed decision

Το authentication/authorization ανήκει στο:

```
auth-service -> auth-postgres
```

Το `auth-service` είναι υπεύθυνο για:

Responsibility	Απόφαση
Google login	ναι
Email/password login	ναι
JWT issuing	ναι
Refresh tokens	ναι, απλό implementation
Password hashing	ναι
Role assignment	ναι

Responsibility	Απόφαση
Seed operator account	ναι
Provider admin association	ναι, μέσω logical `provider_id`
User profile management	minimal

Status: `proposed`

SPEC-8B - Login methods

Proposed decision

Υποστηρίζουμε δύο login methods:

Login method	Χρήση
Google OAuth login	κυρίως EV users, αλλά μπορεί να χρησιμοποιηθεί γενικά
Email/password login	EV users, provider admins, operator/demo accounts

Το SRS/εκφώνηση ζητά Google login για EV users, αλλά εμείς προσθέτουμε απλό email/password login για πληρότητα, demo και ευκολία ανάπτυξης. Η εκφώνηση αναφέρει Google login ως ελάχιστη λειτουργία για EV users. Εκφώνηση Εξαμηνιαίας Εργασίας

Status: `proposed`

SPEC-8C - Roles

Proposed decision

Κλειδώνουμε τρεις βασικούς ρόλους:

```
EV_USER
PROVIDER_ADMIN
OPERATOR
```

Role	Περιγραφή
`EV_USER`	τελικός χρήστης ηλεκτρικού οχήματος
`PROVIDER_ADMIN`	διαχειριστής συγκεκριμένου provider
`OPERATOR`	διαχειριστής του saasPlug

Για provider admins, το auth record περιέχει logical association:

```
provider_id
provider_code
```

Δεν υπάρχει cross-service foreign key. Το `provider_id` είναι logical reference προς `provider-registry-service`.

Status: `proposed`

SPEC-8D - Token strategy

Proposed decision

Χρησιμοποιούμε JWT access tokens.

Token	Χρήση
Access token	short-lived JWT για API calls
Refresh token	αποθηκευμένο hashed στο `auth-postgres`
Token claims	`sub`, `email`, `roles`, `provider_id`, `provider_code`, `exp`

Example JWT claims

JSON

```json
{
  "sub": "user-uuid",
  "email": "provider-admin@example.com",
  "roles": ["PROVIDER_ADMIN"],
  "provider_id": "provider-uuid",
  "provider_code": "redPlug",
  "exp": 1718035200
}
```

Status: `proposed`

SPEC-8E - Access control rules

Proposed decision

Feature/API area	`EV_USER`	`PROVIDER_ADMIN`	`OPERATOR`
Search map/catalog	yes	optional yes	yes
View point details	yes	yes	yes
Check status	yes	yes	yes

Feature/API area	`EV_USER`	`PROVIDER_ADMIN`	`OPERATOR`
Make reservation	yes	no	no
View own reservations	yes	no	no
Register provider	no	limited/self-service if enabled	yes
View provider analytics	no	own provider only	all providers
Export usage logs	no	own provider only	all providers
View invoices	no	own provider only	all providers
Pay invoice	no	own provider only	no or admin override only
Issue invoice	no	no	yes
Edit pricing rules	no	no	yes
Trigger provider sync	no	no	yes
View global analytics	no	no	yes

Key rule: provider admins are always scoped to their own provider.

Status: `proposed`

SPEC-8F - Seed/demo users

Proposed decision

For reproducible Docker demo, seed:

User	Role	Purpose
`operator@saasplug.local`	`OPERATOR`	admin/operator demo
`evuser@saasplug.local`	`EV_USER`	email/password EV login demo
`red-admin@saasplug.local`	`PROVIDER_ADMIN`	redPlug admin
`green-admin@saasplug.local`	`PROVIDER_ADMIN`	greenPlug admin
`blue-admin@saasplug.local`	`PROVIDER_ADMIN`	bluePlug admin

Google OAuth can be configured via `.env`, but the email/password demo path ensures the application remains demonstrable even if OAuth credentials are not available on the

evaluation machine.

Status: `proposed`

SPEC-8G - Security boundaries

Proposed decision

Rule	Απόφαση
Passwords	stored hashed, never plaintext
Provider API keys	stored encrypted/obfuscated where feasible in provider-registry DB
Browser direct provider calls	forbidden
Browser direct internal service ports	forbidden
Gateway	validates/routes API access
Services	still enforce role/provider checks, not only gateway
`.env` secrets	never committed
`.env.example`	committed with placeholders

Status: `proposed`

SPEC-8 summary

Πεδίο	Πρόταση
Auth service	`auth-service`
DB	`auth-postgres`
Login	Google OAuth + email/password
Tokens	JWT access + refresh token
Roles	`EV_USER`, `PROVIDER_ADMIN`, `OPERATOR`
Provider scoping	provider admins scoped to own provider
Demo users	seeded
Secrets	`.env`, not committed

Status: `proposed`

SPEC-9 - Frontend UX, Pages & User Flows

Σκοπός του SPEC-9 είναι να κλειδώσουμε τις σελίδες και τις βασικές ροές UI ώστε αργότερα το Codex να υλοποιήσει συγκεκριμένο frontend χωρίς ασάφεια.

SPEC-9A - Frontend app

Proposed decision

Το frontend είναι ένα web app:

```
apps/frontend
```

με role-based views:

Role	UI area
Public/unauthenticated	landing/login/register
EV user	map/search/reservations
Provider admin	provider dashboard/analytics/invoices/export
Operator	global dashboard/providers/pricing/sync/invoices

Status: `proposed`

SPEC-9B - Public/auth pages

Proposed decision

Page	Purpose
`/login`	Google login + email/password login
`/register`	EV user email/password registration
`/provider-register`	provider registration request/basic provider onboarding
`/logout`	logout/session cleanup

Provider registration can either create a pending provider or be operator-approved, depending on final implementation complexity. For MVP, we can allow simple registration and operator activation.

Status: `proposed`

SPEC-9C - EV user pages

Proposed decision

Page	Purpose
`/map` or `/dashboard`	charging point map/search
`/points/:id`	point details, status, reserve action
`/reservations`	own reservation history
`/profile`	basic account info

Map behavior

The map does **not** load all points at once. It loads points by current viewport/bounds:

```
GET /api/catalog/points?minLat=...&maxLat=...&minLng=...&maxLng=...&zoom=...
```

When the user pans/zooms, frontend debounces the request and reloads points for the visible area.

Status: `proposed`

SPEC-9D - Point details UI

Proposed decision

Point details come from the cached/local aggregated catalog.

Normal flow:

```
frontend
  -> api-gateway
  -> catalog-service
  -> catalog-postgres
```

Optional status flow:

```
frontend
  -> api-gateway
  -> status-service
  -> status-postgres cache
  -> provider service only if cache expired
```

Displayed fields

Field	Source
provider name/code	catalog
point name/location	catalog
address	catalog
coordinates	catalog
connector type	catalog
power/capacity	catalog
price or “Price unavailable”	catalog
last catalog sync timestamp	catalog
last known status	status cache
last status checked timestamp	status service
reserve button	frontend logic based on status

Status: `proposed`

SPEC-9E - Reservation UX

Proposed decision

The reservation flow is simple:

1. EV user opens point details.
2. User clicks `Reserve`.
3. User chooses/request duration if supported.
4. Frontend calls `reservation-service`.
5. UI displays success/failure immediately.
6. Reservation appears in `/reservations`.
7. Events propagate to analytics/billing asynchronously.

Reservation states shown to user

State	UI message
success	reservation confirmed + end time
provider timeout	reservation could not be completed

State	UI message
unavailable/reserved	point is not currently available
missing provider response	reservation status unknown; retry later

Status: `proposed`

SPEC-9F - Provider admin pages

Proposed decision

Page	Purpose
`/provider/dashboard`	provider overview KPIs
`/provider/points`	own charging points
`/provider/analytics`	detailed analytics
`/provider/invoices`	issued invoices, unpaid/paid status
`/provider/invoices/:id`	invoice details + pay button
`/provider/export`	CSV export of own usage logs
`/provider/settings`	provider API/config info, limited edit if allowed

Provider limitations

Provider admin:

- cannot issue official invoices,
- cannot edit pricing rules,
- cannot view other providers,
- cannot trigger global sync unless explicitly allowed later.

Status: `proposed`

SPEC-9G - Operator pages

Proposed decision

Page	Purpose
`/operator/dashboard`	global KPIs
`/operator/providers`	provider list/status/config

Page	Purpose
`/operator/analytics`	global analytics + provider comparison
`/operator/catalog`	catalog health/sync status
`/operator/invoices`	all invoices
`/operator/invoices/new`	issue invoice for provider/period
`/operator/pricing`	edit global/per-provider pricing rules
`/operator/sync`	trigger provider/catalog sync
`/operator/system`	simple health/status overview

Status: `proposed`

SPEC-9H - Dashboard freshness indicators

Proposed decision

All dashboard pages show freshness indicators:

Indicator	Example
analytics computed at	`Metrics computed at 14:00`
catalog last sync	`Catalog last synced today 02:00`
status checked at	`Status checked 3 minutes ago`
billing updated at	`Billing updated at 14:10`

This is important because analytics and billing are precomputed/eventually consistent.

Status: `proposed`

SPEC-9I - API route groups from frontend perspective

Proposed decision

Frontend only calls gateway paths:

```
/api/auth/*
/api/catalog/*
/api/status/*
/api/reservations/*
```

```
/api/analytics/*
/api/billing/*
/api/providers/*
```

No frontend call goes to:

```
red-provider-service:port
green-provider-service:port
blue-provider-service:port
external provider API URL
service-internal Docker DNS names
```

Status: `proposed`

SPEC-9J - UX style and complexity

Proposed decision

Keep UI clean and functional, not overdesigned.

Area	Approach
Map	functional map/list hybrid
Tables	sortable/filterable basic tables
Charts	simple line/bar/pie-like charts if easy
Forms	simple validation
Errors	friendly messages, no raw stack traces
Loading states	required
Empty states	required
Mobile responsiveness	basic, not full mobile app

Status: `proposed`

SPEC-9 summary

Πεδίο	Πρόταση
Frontend	one role-based web app
EV UI	map/search/details/reservations

Πεδίο	Πρόταση
Provider UI	dashboard/analytics/invoices/export/settings
Operator UI	global dashboard/providers/pricing/sync/invoices
Map loading	viewport/bounds-based
Point details	catalog cache + optional status cache
Invoices	provider views/pays, operator issues
API access	only through gateway
Freshness indicators	required

Status: `proposed`

Απόφαση που χρειάζεται

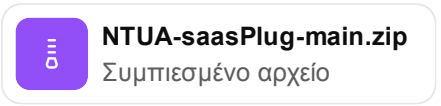
Να κλειδώσουμε τα **SPEC-8** και **SPEC-9** με τις παραπάνω αποφάσεις;

Τα σημεία που κυρίως θέλουν επιβεβαίωση:

SPEC	Θέμα	Πρόταση
SPEC-8	Login	Google + email/password
SPEC-8	Roles	`EV_USER`, `PROVIDER_ADMIN`, `OPERATOR`
SPEC-8	Tokens	JWT + refresh tokens
SPEC-8	Demo users	seeded users
SPEC-9	Map	viewport/bounds-based
SPEC-9	Point details	cached catalog
SPEC-9	Provider invoice actions	view/pay/request only, no issue
SPEC-9	Operator invoice actions	issue invoices
SPEC-9	Gateway-only API	yes

NTUA-saasPlug-main (1…
Συμπιεσμένο αρχείο

Before I send the first promot (00 of audit), is the repo okay now?
Does it have all the files needed in there and in the proper locations?



Είμαστε τώρα έτοιμοι να δώσουμε το audit prompt 00?

NTUA-saasPlug-main.zip
Συμπιεσμένο αρχείο

I installed Docker Desktop and configured it correctly. Codex also worked and did this (everything okay + project creation):

COMPLETED_WITH_WARNINGS

Updated the AI logging workflow and completed direct GitHub project setup for `ntua-el21026/NTUA-saasPlug`. No application code, source files, Docker runtime files, dependencies, microservices, frontend code, or business

Εμφάνιση περισσότερων

Κατάσταση μετά το 00B

Με βάση το **Codex output**, όλα φαίνονται φυσιολογικά και αναμενόμενα:

- το GitHub access πλέον λειτουργεί,
- δημιουργήθηκαν labels/milestones/project/issues #1-#14,
- το `origin` remote διορθώθηκε προς `ntua-el21026/NTUA-saasPlug`,
- ενημερώθηκαν τα AI logging files,
- δεν πειράχτηκε application/source/Docker/runtime logic,
- το επόμενο βήμα είναι πράγματι το `prompt_01_repo_foundation.md`.

Όμως: το **zip που ανέβασες εδώ δεν φαίνεται να είναι το ενημερωμένο zip μετά το 00B**. Το zip που επιθεώρησα δεν περιέχει `docs/ai/`, `docs/specs/SPECIFICATIONS.md`, ούτε το `00B-logging-and-github-setup.md`. Άρα δεν μπορώ να επιβεβαιώσω από το zip την τωρινή κατάσταση. Μπορώ να την αποδεχτώ μόνο βάσει του Codex report που έδωσες.

Πριν ανοίξεις νέα συνομιλία, κάνε ένα νέο zip/export από το **τρέχον repo μετά το 00B**, ώστε να περιλαμβάνει τα updated files.

Τι να κάνεις πριν τη νέα συνομιλία

Στο τοπικό repo, έλεγξε:

Bash

```
git status --short
test -f AGENTS.md
test -f docs/specs/SPECIFICATIONS.md
test -f docs/ai/prompts/prompt_01_repo_foundation.md
test -f docs/ai/codex-runs/000-audit-only.md
test -f docs/ai/codex-runs/00B-logging-and-github-setup.md
test -f docs/ai/codex-runs/_template.md
test -f docs/ai/prompt-log.md
docker --version
```

```
docker compose version
gh auth status
gh repo view ntua-el21026/NTUA-saasPlug --json nameWithOwner,viewerPermission,hasIssuesEn
```

Αν αυτά περνούν, είμαστε πρακτικά έτοιμοι για το Prompt 01.

Συνιστώ να κάνεις ένα commit πριν το Prompt 01:

```
docs: finalize AI logging and GitHub project setup
```

Δεν είναι υποχρεωτικό, αλλά είναι καθαρό checkpoint.

Prompt για νέα συνομιλία ChatGPT

Χρησιμοποίησε το παρακάτω ως πρώτο μήνυμα στη νέα συνομιλία. Επισύναψε εκεί το **νέο zip του repo μετά το 00B**.

Επεξεργασία

We are continuing the NTUA ECE course project **"Technologies of Software Services / Τεχνολογίες Υπηρεσιών Λογισμικού"**. The project is called **saasPlug**.

The discussion can be in Greek, but **all repository files, documentation, code, comments, README files, Codex prompts, and AI logs must be in English**.

I will attach the current repository zip. Your first task is to inspect it deeply and confirm that you understand the full project context before we continue.

Project purpose

saasPlug is a SaaS web application for EV charging-point discovery and reservation. EV users search for charging points from multiple providers and reserve them. Charging infrastructure providers expose their APIs to saasPlug, view usage analytics, receive invoices, pay issued invoices through a mock payment flow, and export their usage logs. The saasPlug operator manages global analytics, providers, pricing rules, sync, and invoices.

The application integrates three hypothetical providers:

- `redPlug`
- `greenPlug`
- `bluePlug`

Each provider has its own OpenAPI YAML specification and schema. Provider services must normalize those APIs into internal DTOs.

Important status

We have already completed the architecture/specification phase in a previous long conversation.

The accepted specifications are consolidated in:

`docs/specs/SPECIFICATIONS.md`

The permanent Codex/agent operating instructions are in:

`AGENTS.md`

These two files are canonical. If anything else conflicts with them, follow `AGENTS.md` and `docs/specs/SPECIFICATIONS.md`.

Accepted architecture summary

You must understand and preserve the following accepted decisions:

1. One role-based frontend web app:
   - React + TypeScript + Vite.
2. Backend:
   - Python FastAPI services.
3. Gateway:
   - Nginx reverse proxy/API gateway.
   - Frontend calls only gateway paths.
   - No browser direct calls to internal services or provider APIs.
4. Runtime:
   - Docker Compose.
5. Databases:
   - PostgreSQL only.
   - Separate PostgreSQL container per stateful service.
   - No MongoDB for MVP.
6. Messaging:
   - RabbitMQ with a topic exchange for background/event flows.
   - RabbitMQ is used for provider sync, catalog aggregation, analytics aggregation, and billing propagation.
   - RabbitMQ is **not** used for live frontend actions, login, map search, status lookup, or reservation provider calls.
7. No dedicated orchestrator service:
   - Use local orchestration inside responsible services.
   - Use event choreography for asynchronous propagation.
8. Core services:
   - `auth-service`
   - `provider-registry-service`
   - `red-provider-service`
   - `green-provider-service`
   - `blue-provider-service`

- `catalog-service`
- `status-service`
- `reservation-service`
- `analytics-service`
- `billing-service`

9. Provider services:
    - Stateful.
    - Each has its own PostgreSQL database/container.
    - They store provider-local point snapshots, raw payloads, sync runs, and changes.
10. Catalog:
    - `catalog-service` owns the aggregated normalized charging-point catalog.
    - Map loading is viewport/bounds-based, not “load all points”.
    - Point details come from the cached/local aggregated catalog.
11. Status:
    - `status-service` owns 5-minute status cache and status logs.
    - Live provider status calls happen only when cache is expired or explicitly needed.
12. Reservation:
    - `reservation-service` owns reservation truth.
    - Reservation calls are real-time synchronous provider calls.
    - No queue before reservation provider call.
13. Analytics:
    - `analytics-service` consumes events and stores raw `usage_events` idempotently.
    - Dashboards read precomputed aggregate/cache tables.
    - Aggregation is hourly + daily rollups + manual refresh.
    - Provider analytics include realistic KPIs: searches, point views, status checks, successful/failed reservations, conversion rates, top points, status distribution, charges/fees, data freshness, CSV export.
    - Operator analytics include global KPIs, provider comparison, trends, billing overview, catalog health, and manual refresh.
14. Billing:
    - `billing-service` owns pricing rules, charges, invoices, and mock payments.
    - Charges come mainly from successful reservations and optionally point views.
    - Failed reservations are not charged by default.
    - Pricing supports global default and per-provider overrides.
    - Only the operator/admin can issue official invoices.
    - Providers can view and pay issued invoices and may optionally request issuance.
    - Payment is mock and full-invoice only.

- No partial payments, refunds, real payment gateway, complex tiers, subscriptions, or tax/accounting compliance.

15. Auth:
    - `auth-service` supports Google OAuth and email/password login.
    - JWT access tokens + refresh tokens.
    - Roles:
        - `EV_USER`
        - `PROVIDER_ADMIN`
        - `OPERATOR`
    - Provider admins are scoped to their own provider.
16. Documentation:
    - Final diagrams go under:

      `docs/diagrams/final/`
    - Intermediate diagrams remain historical and may contain obsolete names/routes.
    - README files are required in all major folders/services/apps/packages/docs/deploy/scripts.
17. AI documentation:
    - Official AI logs are unavailable.
    - We use repository-based manual AI logging under:

      `docs/ai/`
    - Every Codex run must update:

      `docs/ai/prompt-log.md`
      `docs/ai/codex-runs/<run-id>-<run-name>.md`
    - Run reports must distinguish:
        - Invocation Prompt: exact text pasted into Codex.
        - Canonical Task Prompt: prompt file under `docs/ai/prompts/`, when used.
18. GitHub project management:
    - Codex already performed the GitHub setup in a previous run.
    - GitHub repo target:

      `ntua-el21026/NTUA-saasPlug`
    - GitHub Project:

      `saasPlug Implementation`
    - Issues #1-#14 correspond to prompts 01-14.
    - If future GitHub mutations fail, Codex must report the failure clearly, not create Markdown fallback project-management artifacts unless explicitly requested.

What has happened so far

A prior Codex audit-only run returned:

`READY_WITH_WARNINGS`

It created/updated:

```
docs/ai/codex-runs/000-audit-only.md
docs/ai/prompt-log.md
```

Then a setup run, Prompt 00B, returned:

```
COMPLETED_WITH_WARNINGS
```

It updated AI logging workflow files and completed direct GitHub project setup.

Prompt 00B changed:

```
AGENTS.md
docs/ai/README.md
docs/ai/prompts/README.md
docs/ai/codex-runs/README.md
docs/ai/codex-runs/_template.md
docs/ai/prompts/prompt_01_repo_foundation.md
docs/ai/prompts/prompt_00b_logging_and_github_setup.md
docs/ai/prompt-log.md
docs/ai/codex-runs/00B-logging-and-github-setup.md
```

It also:

- verified `gh auth status`,
- verified repo access to `ntua-el21026/NTUA-saasPlug`,
- changed local `origin` remote to:

  `https://github.com/ntua-el21026/NTUA-saasPlug.git`
- created all requested labels,
- created three milestones,
- created GitHub Project `saasPlug Implementation`,
- created issues #1-#14 and added them to the project.

Docker Desktop has now been installed and configured correctly.

Files you must inspect carefully from the attached repo zip

Before giving any implementation prompt, inspect at least:

```
AGENTS.md
README.md
docs/specs/SPECIFICATIONS.md
docs/specs/README.md

docs/ai/README.md
docs/ai/prompt-log.md
docs/ai/prompts/README.md
docs/ai/prompts/prompt_00b_logging_and_github_setup.md
docs/ai/prompts/prompt_01_repo_foundation.md
```

```
docs/ai/codex-runs/README.md
docs/ai/codex-runs/_template.md
docs/ai/codex-runs/000-audit-only.md
docs/ai/codex-runs/00B-logging-and-github-setup.md

docs/assignment/
docs/assignment/SRS Εξαμηνιαίας Εργασίας.docx
docs/assignment/Εκφώνηση Εξαμηνιαίας Εργασίας.pdf
docs/assignment/Διαγράμματα SRS.vpp
docs/assignment/providers/
docs/assignment/providers/Γενική περιγραφή APIs παρόχων φόρτισης.pdf
docs/assignment/providers/yaml/redPlug.yaml
docs/assignment/providers/yaml/greenPlug.yaml
docs/assignment/providers/yaml/bluePlug.yaml
docs/assignment/providers/postman_collections/

docs/lectures/
docs/diagrams/
docs/diagrams/intermediate/
docs/diagrams/final/
```

Also inspect the current repository structure and identify whether Prompt 01 is still the correct next action.

Your task in this new ChatGPT conversation

Do not implement anything yourself.

First, inspect the attached repo zip and summarize:

1. Whether the repo appears ready for Prompt 01.
2. Whether the AI logging structure is now correct.
3. Whether the accepted specs and AGENTS.md are present and coherent.
4. Whether there are any blockers before running Prompt 01.
5. Whether Docker/GitHub setup appears already handled based on the run logs.
6. Whether the repository zip seems current after Prompt 00B.

Then, if everything is acceptable, give me the exact Codex invocation prompt for Prompt 01.

The final answer should be:

1. A concise but complete confirmation that you understood the project, specs, repository state, and workflow.
2. Any warnings/blockers.
3. The exact short invocation prompt to paste into Codex for Prompt 01.

The Prompt 01 invocation should follow this model:

```
Read and execute docs/ai/prompts/prompt_01_repo_foundation.md.
Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.
Use docs/ai/codex-runs/_template.md for the run report.
Update docs/ai/codex-runs/001-repo-foundation.md and docs/ai/prompt-log.md.
Record both the exact invocation prompt and the canonical task prompt file/
Do not implement business logic, services, frontend screens, Docker runtime
```

Συμπέρασμα για την τρέχουσα κατάσταση

Βάσει του Codex output, είσαι **έτοιμος να περάσεις στη νέα συνομιλία** και μετά στο Prompt 01.

Η μόνη επιφύλαξη είναι ότι το zip που ανέβασες εδώ δεν φαίνεται να είναι το ενημερωμένο μετά το 00B. Στη νέα συνομιλία, ανέβασε **fresh zip** από το τοπικό repo μετά το 00B.

NTUA-saasPlug-main (1…
Συμπιεσμένο αρχείο

Δες ξανά το zip.

Όχι. Το νέο zip που ανέβασες **δεν είναι το ενημερωμένο repo μετά το 00B**. Είναι παλαιότερη κατάσταση.

Τι βρήκα στο zip

Στο zip υπάρχουν:

```
AGENTS.md
docs/specs/SPECIFICATIONS.md
docs/ai/prompt-log.md
docs/ai/prompts/CODEX_PROMPTS_README.md
docs/ai/prompts/prompt_01_repo_foundation.md
docs/ai/codex-runs/codex_run_template.md
```

Αλλά λείπουν τα αρχεία που, σύμφωνα με το Codex output, δημιουργήθηκαν/ενημερώθηκαν από το 00B:

```
docs/ai/README.md
docs/ai/prompts/README.md
docs/ai/prompts/prompt_00b_logging_and_github_setup.md
docs/ai/codex-runs/README.md
docs/ai/codex-runs/_template.md
docs/ai/codex-runs/000-audit-only.md
docs/ai/codex-runs/00B-logging-and-github-setup.md
```

Επίσης υπάρχουν ακόμα τα παλιά ονόματα:

```
docs/ai/prompts/CODEX_PROMPTS_README.md
docs/ai/codex-runs/codex_run_template.md
```

ενώ μετά τις διορθώσεις/00B θα έπρεπε να υπάρχουν:

```
docs/ai/prompts/README.md
docs/ai/codex-runs/_template.md
```

Συμπέρασμα

Το zip είναι **stale / pre-00B**. Δεν πρέπει να το χρησιμοποιήσουμε για τη νέα συνομιλία ή για να συνεχίσουμε με Prompt 01.

Πιθανή αιτία: οι αλλαγές του Codex 00B είναι ακόμα **uncommitted local changes** στο `/home/peppasmich/NTUA-saasPlug`, ενώ το zip μάλλον προήλθε από GitHub download ή από παλιό φάκελο. Το ίδιο το Codex είπε ότι το commit after run ήταν `not committed`.

Τι να κάνεις τώρα

Στο πραγματικό local repo όπου έτρεξε το Codex:

Bash

```
cd /home/peppasmich/NTUA-saasPlug
git status --short
```

Πρέπει να δεις τα changed/untracked AI files του 00B. Μετά κάνε ένα checkpoint commit:

Bash

```
git add AGENTS.md docs/ai
git commit -m "docs: finalize AI logging and GitHub project setup"
```

Μετά φτιάξε νέο zip **από αυτόν τον τοπικό φάκελο**, όχι από GitHub download, εκτός αν πρώτα κάνεις push.

Για να επιβεβαιώσεις πριν το zip:

Bash

```
test -f docs/ai/README.md
test -f docs/ai/prompts/README.md
test -f docs/ai/prompts/prompt_00b_logging_and_github_setup.md
test -f docs/ai/codex-runs/README.md
test -f docs/ai/codex-runs/_template.md
test -f docs/ai/codex-runs/000-audit-only.md
test -f docs/ai/codex-runs/00B-logging-and-github-setup.md
```

Αν όλα περάσουν, τότε κάνε νέο zip και ανέβασέ το.

Τελική κατάσταση

Έλεγχος	Status
Codex 00B output	φαίνεται σωστό

Έλεγχος	Status
Uploaded zip	όχι ενημερωμένο
Ready for new ChatGPT continuation with this zip	όχι
Ready after fresh local zip/commit	ναι

Στη νέα συνομιλία να βάλω μόνο το .zip, σωστά;

Όχι μόνο το `.zip`.

Στη νέα συνομιλία πρέπει να βάλεις:

1. **Το φρέσκο `.zip` του repo μετά το 00B**
   - από το local repo `/home/peppasmich/NTUA-saasPlug`
   - όχι GitHub download αν δεν έχεις κάνει push
   - πρέπει να περιέχει τα updated `docs/ai/*`, `00B-logging-and-github-setup.md`, `_template.md`, κτλ.
2. **Το handoff prompt που σου έδωσα**
   - ως πρώτο μήνυμα στη νέα συνομιλία
   - αυτό δίνει όλο το context, τις αποφάσεις SPEC-0 έως SPEC-11, τι να διαβάσει, και τι τελική απάντηση περιμένεις.

Άρα η σωστή εκκίνηση είναι:

```
[Paste the full handoff prompt]

Attach:
NTUA-saasPlug-main.zip
```

Μην ανεβάσεις μόνο το zip χωρίς prompt, γιατί η νέα συνομιλία μπορεί να μη γνωρίζει ακριβώς:

- ότι πρέπει να διαβάσει `AGENTS.md` και `SPECIFICATIONS.md`,
- ότι το repo έχει περάσει audit 00 και 00B,
- ότι επόμενο βήμα είναι Prompt 01,
- ότι η απάντηση που θέλεις είναι επιβεβαίωση + exact Codex invocation prompt.

Πριν φτιάξεις το zip, έλεγξε στο local repo:

Bash

```bash
test -f docs/ai/README.md
test -f docs/ai/prompts/README.md
test -f docs/ai/prompts/prompt_00b_logging_and_github_setup.md
test -f docs/ai/codex-runs/README.md
test -f docs/ai/codex-runs/_template.md
```

```
test -f docs/ai/codex-runs/000-audit-only.md
test -f docs/ai/codex-runs/00B-logging-and-github-setup.md
```

Αν αυτά υπάρχουν, τότε κάνεις zip και ξεκινάς τη νέα συνομιλία με **prompt + zip**.

Μπες στο repo μου και έλεγξε το.

Έγινε λήψη της απάντησης ης εφαρμογής

Ναι. Μπήκα στο GitHub repo `ntua-el21026/NTUA-saasPlug` και το έλεγξα. Η κατάσταση είναι πλέον **σωστή και αναμενόμενη μετά το 00B**.

Έλεγχος κατάστασης

OK

Περιοχή	Κατάσταση
GitHub repo	Υπάρχει ως `ntua-el21026/NTUA-saasPlug`, private, με admin/maintain/push permissions
`AGENTS.md`	Υπάρχει και περιέχει τις τελικές operating rules
`SPECIFICATIONS.md`	Υπάρχει και είναι το accepted baseline
AI logging workflow	Υπάρχει και ξεχωρίζει invocation prompt / canonical task prompt
`prompt_01_repo_foundation.md`	Υπάρχει και είναι σωστά περιορισμένο σε repository foundation
GitHub issues	Υπάρχουν issues #1-#14 για Prompt 01 έως Prompt 14
GitHub project setup	Το 00B log δηλώνει ότι δημιουργήθηκε project και προστέθηκαν τα 14 issues
Root README / `.env.example` / scaffold folders	Λείπουν ακόμη, αλλά αυτό είναι **αναμενόμενο** και ανήκει στο Prompt 01

Σημαντικά ευρήματα

Το `AGENTS.md` έχει πλέον τις κρίσιμες τελικές οδηγίες: mandatory reading, canonical service/container names, data ownership, provider strategy, map bounds loading, point details from catalog, status cache, reservation rule, RabbitMQ boundaries, no dedicated orchestrator, analytics/billing/auth/frontend rules. Ενδεικτικά, ορίζει ότι το frontend δεν πρέπει να καλεί internal service ports ή external provider APIs απευθείας, και ότι η

accepted architecture είναι React/Vite + FastAPI + Nginx + RabbitMQ + separate PostgreSQL container per stateful service.

AGENTS

Το `SPECIFICATIONS.md` είναι παρόν και δηλώνει ότι είναι το canonical specification baseline για την υλοποίηση, με σκοπό να αποτρέψει architecture drift κατά την ανάπτυξη. Περιλαμβάνει τα non-negotiable rules για provider API handling, gateway-only frontend access, database ownership, RabbitMQ boundaries, no dedicated orchestrator και manual AI logs.

SPECIFICATIONS

Το AI logging είναι πλέον σωστά διαμορφωμένο. Το `docs/ai/README.md` εξηγεί ρητά τη διάκριση ανάμεσα σε **Invocation Prompt** και **Canonical Task Prompt**, και περιγράφει τη ροή `short invocation prompt -> Codex reads canonical prompt file -> Codex executes task -> Codex writes run report -> Codex updates prompt-log`.

README

Το `_template.md` έχει τα απαιτούμενα sections για κάθε run report: metadata, invocation prompt, canonical task prompt, canonical prompt content, sources read, files changed, commands run, tests/checks, warnings/blockers, GitHub actions, final status και next action.

_template

Το `prompt-log.md` έχει πλέον rows για `000`, `00B` και pending `001`, με canonical prompt file και run report path. Αυτό είναι ακριβώς το logging μοντέλο που θέλαμε.

prompt-log

Το `prompt_01_repo_foundation.md` είναι σωστό για το επόμενο βήμα: δεν ζητά application logic, APIs, DB models, Docker runtime ή frontend screens· ζητά μόνο foundation folders, README coverage, `.env.example`, `.gitignore`, diagrams/AI paths και run log.

prompt_01_repo_foundation

Τα GitHub issues για Prompt 01-14 υπάρχουν. Ενδεικτικά, το issue #1 είναι `Prompt 01 - Repository foundation`, και η αναζήτηση επέστρεψε επίσης τα issues για Prompt 02 έως Prompt 14.

Prompt 01 - Repository foundati…

Prompt 11 - Billing service

Warnings

Δεν βλέπω blocker.

Υπάρχουν μόνο αναμενόμενα pre-foundation gaps:

Warning	Γιατί δεν είναι πρόβλημα
Root `README.md` είναι ακόμη ελάχιστο	Το Prompt 01 θα το ενημερώσει
`.env.example` λείπει	Το Prompt 01 πρέπει να το δημιουργήσει
`apps/`, `services/`, `packages/`, `deploy/`, `scripts/` μάλλον δεν υπάρχουν ακόμη	Το Prompt 01 είναι ακριβώς για αυτό
Application code δεν υπάρχει	Αναμενόμενο πριν την πραγματική υλοποίηση
Docker runtime δεν υπάρχει	Θα αρχίσει στο Prompt 02, όχι τώρα

Verdict

READY_FOR_PROMPT_01

Το repo είναι πλέον σε σωστή κατάσταση για να συνεχίσουμε με το πρώτο πραγματικό Codex implementation step.

Exact Codex invocation prompt για Prompt 01

```
Read and execute docs/ai/prompts/prompt_01_repo_foundation.md.
Follow AGENTS.md and docs/specs/SPECIFICATIONS.md.
Use docs/ai/codex-runs/_template.md for the run report.
Update docs/ai/codex-runs/001-repo-foundation.md and docs/ai/prompt-log.md.
Record both the exact invocation prompt and the canonical task prompt file/content
Do not implement business logic, services, frontend screens, Docker runtime, databa
```